CSS 311 — Parallel and Distributed Computing
ASSIGNMENT – I

Topic: PageRank Algorithm

Name: _____

Roll No: _____

Q4 and Q9 (source code + program output) are not reproduced in this PDF — see the accompanying ZIP (c/, outputs/) and the Code tab of the web UI. Only Q1,2,3,5 (Module-I) and Q6,7,8,10 (Module-II) are handwritten, as permitted by the assignment.

MODULE – I

Q1. Objective and Description

- **Objective:** Compute importance/rank of nodes in a directed graph using PageRank, and study its parallelization using OpenMP.
- **What:** PageRank assigns a rank to each node from the ranks of its incoming neighbours.
- **Why:** Repeated computation on large graphs is costly.
- **Where:** Web/search ranking and link analysis.

Q2. Algorithm / Pseudocode

```
Initialize PR[i] = 1/N

repeat
    dangling = sum PR[j], for outdeg(j) = 0

    for each node i:
        newPR[i] = (1-d)/N + d*dangling/N
        for each j -> i:
            newPR[i] += d*PR[j]/outdeg(j)

    error = sum |newPR[i] - PR[i]|
    PR = newPR

until error < eps
```

Q3. Solution Demonstration

Two small test cases, computed with the same recurrence as Q2 (d = 0.85). Each shows the graph, one worked iteration, and the converged result.

Test Case 1 — 3 nodes, no dangling node

```
Graph: 0->1, 0->2, 1->2, 2->0        N=3, d=0.85
outdeg: 0:2  1:1  2:1
Initial PR = 1/3 = 0.3333 each  (no dangling node)

Iteration 1:
  PR[0] = 0.05 + 0.85*(PR[2]/1)             = 0.3333
  PR[1] = 0.05 + 0.85*(PR[0]/2)             = 0.1917
  PR[2] = 0.05 + 0.85*(PR[0]/2 + PR[1]/1)   = 0.4750
  error = 0.2833

Converged (28 iters): PR = (0.3878, 0.2148, 0.3974)
```

Test Case 2 — 4 nodes, node 3 dangling (outdeg=0)

```
Graph: 0->1, 1->2, 2->0, 2->3        N=4, d=0.85
outdeg: 0:1  1:1  2:2  3:0 (dangling)
Initial PR = 1/4 = 0.25 each ; dangling = PR[3] = 0.25

Iteration 1:  base=(1-d)/N=0.0375   dangling term=d*0.25/4=0.053125
  PR[0] = 0.0375 + 0.85*(PR[2]/2) + 0.053125 = 0.1969
  PR[1] = 0.0375 + 0.85*(PR[0]/1) + 0.053125 = 0.3031
  PR[2] = 0.0375 + 0.85*(PR[1]/1) + 0.053125 = 0.3031
  PR[3] = 0.0375 + 0.85*(PR[2]/2) + 0.053125 = 0.1969
  error = 0.2125

Converged (32 iters): PR = (0.2138, 0.2646, 0.3079, 0.2138)
```

Q5. Time Analysis (Serial)

N = vertices, E = edges, I = iterations. Implementation uses an in-link **adjacency list (CSR)** — verified in c/serial_pagerank.c — not a matrix.

```
Per iteration = O(N + E)
Total         = O(I . (N + E))
Space         = O(N + E)
```

Actual measured results (outputs/serial_output.txt):

N	E	Iterations	Time (sec)
1000	5994	13	0.000120
10000	59995	13	0.001672
100000	599992	13	0.020366

MODULE – II (Parallel — OpenMP)

Q6. Identification of Parallelizable Blocks

```
PageRank iteration
       |
       v
compute newPR[i], i = 0..N-1
       |
       v
   PARALLEL
```

- **Parallel:** newPR[i] for each i — independent across i within one iteration.
- **Shared:** PR[], graph / in-link data (read-only during the loop).
- **Private:** loop variable i, local sum.
- **Reduction:** dangling, error.
- **Synchronization:** barrier before next iteration.

Note: only the per-node update within one iteration is parallel — the iteration-to-iteration dependency is sequential (newPR must fully replace PR before the next iteration starts). The algorithm as a whole is **not** fully parallel.

Q7. OpenMP Pseudocode

```
Initialize PR[i] = 1/N

repeat
    parallel for + reduction:
        dangling = sum PR[j], for outdeg(j) = 0

    parallel for:
        for each i: compute newPR[i]
            (reads PR[j] of in-neighbours only)

    barrier

    parallel for + reduction:
        error = sum |newPR[i] - PR[i]|
    PR = newPR
```

```
until error < eps
```

Q8. Solution Demonstration (OpenMP)

Same graph as Q3 Test Case 2 (4 nodes). Static schedule, 2 threads:

```
Nodes: 0 1 2 3

Thread 0 -> {0, 1}
Thread 1 -> {2, 3}
        |
     barrier
        |
  next iteration
```

Result: PR = (0.2138, 0.2646, 0.3079, 0.2138), 32 iterations — **identical** to the serial result (same computation, divided across threads; verified in outputs/openmp_output.txt on the assignment's original 6-node code-test graph).

Q10. Time Analysis (OpenMP)

```
N=vertices, E=edges, I=iterations, P=threads

Serial:      O(I . (N+E))
Parallel:  ~ O(I . (N+E) / P) + sync/reduction overhead
Speedup:     S = T_serial / T_parallel
Efficiency:  Eff = S / P
```

Same three input sizes as Q5 (different test cases):

N	E	Complexity
1000	5994	O(I.(N+E))
10000	59995	O(I.(N+E))
100000	599992	O(I.(N+E))

Actual measured results, N = 100000, outputs/openmp_output.txt:

Threads	Time (sec)	Speedup
1	0.018196	1.00
2	0.018053	1.01
4	0.019645	0.93
8	0.019395	0.94

Measured speed-up is hardware-dependent (this run: 1 logical CPU core available).